グラビティシューター

プログラム説明資料



日本工学院専門学校ゲームクリエイター科　4年制28卒予定
末廣 航一

作品概要

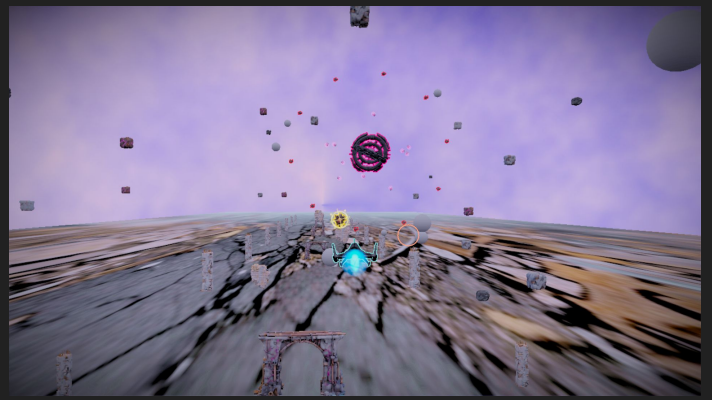


ジャンル：3D奥スクロール・レールシューティング

制作時期：2026年5月～（制作途中 / 約5ヶ月）

開発環境・API：
- C++20 / Visual Studio 2026
- DirectX 12（描画）/ XAudio2（サウンド）/ Windows SDK 10.0.26100

外部ライブラリ：
- Assimp（3Dモデル読み込み）
- DirectXTex（テクスチャ・画像読み込み）
- nlohmann json（JSONデータのパース・保存）
- Dear ImGui（リアルタイムデバッグUI）



担当箇所：プログラム全般（企画・設計・実装）
※効果音・BGMと、一部3Dモデル・UI素材はフリーおよびAI生成素材を使用

ゲームコンセプトと技術目標：
- 『DEATH STRANDING』のような重厚な空気感と、『グラビティデイズ』のような「周囲の物を重力で引き寄せてぶつける」操作感を意識した3D重力シューティング。
- 画面内に多数のガレキや破片が飛び交い、十数種類のポストプロセスを適用した環境下でも、CPU負荷を抑えて**常時60FPS（16.6ms）**で安定動作させることを目標として開発。

2D/3D重み付けスコアリングと透過を防ぐ射線遮蔽判定

【要件と課題】多数の敵機や浮遊ガレキが入り乱れる3D空間において、プレイヤーが意図した敵へ直感的に照準を吸着させつつ、ビル等の遮蔽物の裏にいる見えない敵への透過ロックオンや、遠方の敵への誤吸着を防止する必要があった。

【比較と選定理由】画面中央からの2D距離のみでは画面奥の遠い敵を誤認識し、3D距離のみでは画面端の敵を不自然に吸着してしまう。そこで**2D照準距離と3D深度距離を重み付け合算したスコアリング式**と、**Raycastによる射線遮蔽（Line of Sight）判定**を組み合わせたハイブリッド方式を採用した。

【実装の工夫と代償】

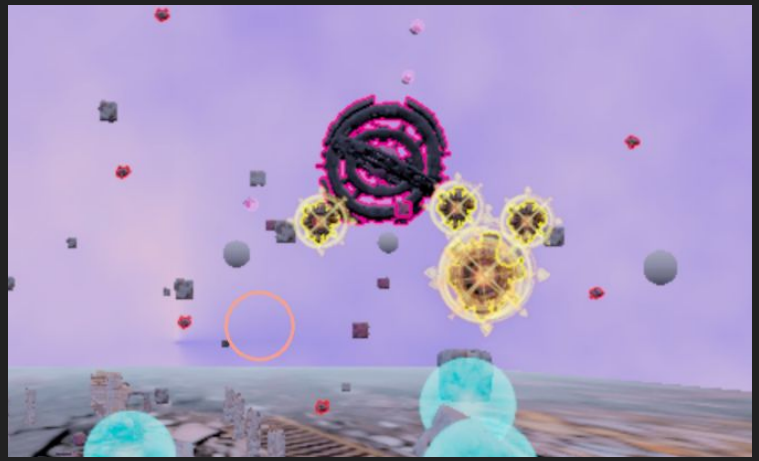
初回同期・以降非同期の遮蔽判定（Amortization）：初回検出時は即座に同期Raycastを実行して壁裏への一瞬の透過表示を防止。追尾継続中は0.1秒ごとの非同期Raycastに切り替えることで、毎フレームの衝突判定によるCPU負荷を分散させた。

重み付けスコア：`score = std::sqrt(dist2DSq) * weight2D_ + dist3D * weight3D_;` により、照準近傍かつ手前の敵を最優先でロックオン。

【定量的な成果】

遮蔽物が多いステージでも誤作動せず、プレイヤーの視線誘導に沿った自然で手応えのあるマルチロックオン操作を実現した。

```cpp
// PlayerTargetingComponent.cpp : 2D/3D重み付け走査と初回同期遮蔽判定
float score = std::sqrt(dist2DSq) * weight2D_ + dist3D * weight3D_;
if (score < bestScore) {
    if (!cache.hasCheckedOnce) {
        // 初回は同期Raycastで遮蔽を確定し、壁裏敵の一瞬の透過ロックを防止
        RaycastHit hitInfo{};
        bool hit = collisionManager->Raycast(ray, hitInfo, dist3D + 10.0f);
        cache.canSee = (!hit || hitInfo.distance >= dist3D - 1.0f);
        cache.hasCheckedOnce = true;
    } // 2回目以降は0.1秒間隔の非同期RaycastでCPU負荷を分散
}
```



戦術ステートマシンと自機移動ベクトルに基づく未来位置予測

【要件と課題】単調な接近と射撃の繰り返しを防ぎ、3D空間を活かした立体的な戦闘と、プレイヤーの回避行動を促す緊迫感のある攻防を構築する必要があった。

【比較と選定理由】単一ループ内での場当たり的な条件分岐を排し、役割ごとの振る舞い（特攻機、定点機、狙撃機）と行動フェーズ（接近➔ 交戦 ➔ 特攻/離脱）を明確に分離した階層型ステートマシンを採用した。

【実装の工夫と代償】

未来予測射撃（Predictive Lead Shooting）：スナイパー型の敵機において、弾速と距離から到達時間を計算し、自機の移動ベクトルを乗じ**未来予測座標**（predictedTarget = playerPos + playerVel * travelTime ）内へ偏差射撃を実行。

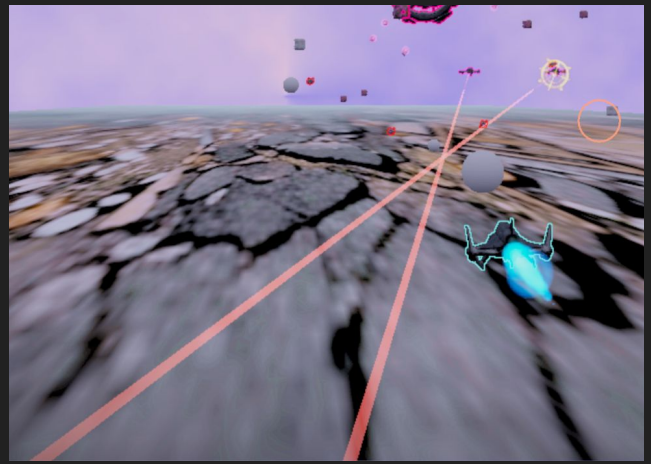
照準予告と回避猶予（Telegraph）：射撃直前まで赤色レーザーシリンダーで狙いを定めた後、最終フェーズで射線を固定して発射することで、プレイヤーに「見てから回避する」猶予と手応えを設計。

【定量的な成果】

敵の役割分担が明確化し、直進し続けると狙撃され、静止していると特攻されるメリハリのある戦闘ゲームプレイを構築した。

```
// RailShooterEnemyComponent.cpp : 弾丸到達時間と未来予測座標の算出
float dist = std::sqrt(dx * dx + dy * dy + dz * dz);
float travelTime = std::clamp(dist / (std::max)(bulletSpeed_, 1.0f), 0.0f, 1.2f);

// 自機の現在位置と移動速度ベクトルから未来予測座標を逆算
Vector3 predictedTarget = { playerPos.x + playerVel.x * travelTime,
                            playerPos.y + playerVel.y * travelTime,
                            playerPos.z + playerVel.z * travelTime };
Vector3 aimDir = Normalize(predictedTarget - myPos);
```



ExecuteIndirect による完全GPU駆動カリング

【要件と課題】 前作までにインスタンシング（Batch描画）は導入済みであり、ドローコール自体の削減は行えていた。しかし本作で大量のガレキや背景が密集するシーンを構築した際、**CPU側での視錐台カリング計算と、毎フレームの可視バッファ構築・転送処理**がCPUを圧迫。ゲームロジックの処理時間を圧迫し、フレームレート低下の原因となった。

【比較と選定理由】 CPU側でマルチスレッド化してカリングを行う手法も検討したが、CPU-GPU間のバッファ転送負荷や同期オーバーヘッドは解消できない。そこで、可視判定から描画引数の動的生成までをすべてGPU側で完結させる**ExecuteIndirectによる完全GPU駆動パイプライン**を自作・導入した。

【実装の工夫と代償】 GPUCulling.CS.hlsl 内で視錐台6平面との球判定を行い、可視オブジェクトのインデックスを`InterlockedAdd` でアトミックに加算して描画引数バッファを動的生成。境界部分のチラつきを防ぐため10%の保守的マージン（Conservative Culling）を設けた。

【定量的な成果】
CPUからの描画コマンド発行をExecuteIndirect の**1回**に集約。CPU側で行っていた可視判定ループとバッファ転送処理を排除し、描画前処理負荷を大幅に低減した。

```
// GPUCulling.CS.hlsl : 視錐台カリングと描画引数のGPUアトミック生成
if (isVisible) {
    uint visibleIndex = 0;
    // D3D12_DRAW_INDEXED_ARGUMENTS の InstanceCount をGPU側で加算
    InterlockedAdd(CommandBuffer[1], 1, visibleIndex);
    OutputInstances[visibleIndex] = outInst;
// 可視インスタンスを行列化格納
}
```

仮想インスタンスによるデータ管理と初期化時プール事前確保

【要件と課題】 空間に1万個規模のガレキが存在する本作において、すべてを通常のGameObject（コンポーネント付きの実体）として生成・保持すると、メモリ消費と毎フレームのUpdate走査負荷でCPUが破綻する。また、ゲームループ中に動的確保（`new`/`delete`）を繰り返すと、メモリ断片化や処理スパイクの原因となる。

【比較と選定理由】 全オブジェクトの常時実体化を廃止。通常時は座標データのみを保持する「仮想インスタンス（VirtualInstance）」として扱い、プレイヤーの重力引き寄せや当たり判定が必要になった瞬間のみ、初期化時に事前確保したプールから**実体化（Promote）**させるハイブリッドアーキテクチャを採用した。

【実装の工夫と代償】
初期化時の事前確保（ObjectPool）：実体化に必要な最大数分だけシーン初期化フェーズで一括事前確保（Pre-allocation）し、ゲームループ中の動的アロケーションをゼロ化。Handleと世代（Generation）管理により不正アクセスを防止。
動的昇格・降格（Promote / Demote）：インタラクション発生時にPromote()でプールから実体を割り当て、投擲や離脱後はDemote()でプールへ即座に返却して仮想データへ戻す疎結合なライフサイクルを構築。

【定量的な成果】
1万個のガレキが空間に存在しつつ、CPUのゲームオブジェクト走査負荷とメモリ確保によるスパイクを排除し、重力アクションの安定した処理環境を実現した。

```
// VirtualEntityManagerComponent.h : 仮想データと事前確保プールの連携
    struct VirtualInstance {
        int id;
        Vector3 position, rotation, scale;
        bool isPromoted = false;
        ObjectPool<GameObject>::Handle promotedHandle;
      // 事前確保プールへのハンドル
    };
    // 重力干渉時のみ実体を割り当て、投擲・回収後は即座にプール返却
    std::shared_ptr<GameObject> Promote(int id);
    void Demote(int id);
```

多重ポストプロセスに伴う状態管理と保守性の課題

【要件と課題】 空間歪み、Bloom、被写界深度（DoF）、輪郭抽出など十数個の描画パスが連携する構成において、DirectX12のリソースバリア（状態遷移）を手動管理していたため、遷移指定の漏れによる警告や不具合の原因となっていた。

【比較と選定理由】 個別パスごとに手動でバリアを記述する手法は、描画順序の入れ替えや新規エフェクト追加のたびに修正範囲が広がり保守性が下がる。そこで、パス間の依存関係からエンジンが実行順序と適切なバリア発行タイミングを自動解決する**RenderGraph**アーキテクチャを自作・導入した。

【実装の工夫と代償】 GPUCulling.CS.hlsl 内で視錐台6平面との球判定を行い、可視オブジェクトのインデックスを`InterlockedAdd` でアトミックに加算して描画引数バッファを生成。境界部分のチラつきを防ぐため、10%の保守的マージン（Conservative Culling）を設けた。

【定量的な成果】
バリア不整合に起因する警告やクラッシュを解消。描画順序の変更や新規ポストエフェクトの追加を安全に行える環境を整えた。

```cpp
// RenderGraph.cpp : 状態追跡によるリソースバリアの自動計算と発行
for (const auto& usage : pass.GetResourceUsages()) {
    // 直前の状態を取得（未登録なら COMMON）
    D3D12_RESOURCE_STATES currentState = resourceStates_[usage.resource];

    // 要求ステートと異なる場合のみ、最小限のバリアを自動生成
    if (currentState != usage.state) {
        barriers.push_back(CreateTransitionBarrier(
            usage.resource, currentState, usage.state));
        resourceStates_[usage.resource] = usage.state; // 状態キャッシュを更新
    }
}
commandList->ResourceBarrier(barriers.size(), barriers.data()); // 一括発行
```

依存関係の自動解決と Pre/Post UIレイヤー分離

【要件と課題】 多重エフェクト適用時、背景の3D世界にはブラーや歪みを適用しつつ、プレイヤーのHUD（照準レティクル・HP）は影響を受けずに鮮明に表示される描画順序制御が必要であった。

【比較と選定理由】 単一のポストプロセスチェーンではUIと3Dシーンの分離が煩雑になる。RenderGraph上で「Pre-UIパス（3D世界のみ）」と「Post-UIパス（画面全体）」をレイヤー分離し、DAG（有向非巡回グラフ）のトポロジカルソートで自動解決する設計を採用した。

【実装の工夫と代償】 各パスはIRenderPassを継承し、`Setup` で入出力リソースを宣言する形式とした。直前のリソース状態をエンジン側で追跡し、必要な箇所にのみ最小限のResourceBarrierを自動発行する仕組みを構築。

【定量的な成果】
描画順序の不整合を防止。背景にDoFや歪みが適用されている状態でも、手前の照準レティクルが鮮明に表示され、視認性の高いUI表示を実現した。

```
// 宣言型レンダーパス（MainOpaquePass.cpp）
void MainOpaquePass::Setup(RenderGraphBuilder& builder, const RenderContext& rc) {
        builder.WriteRenderTarget(rc.mainColorTarget); // 書き込みリソースを宣言
        builder.WriteDepthStencil(rc.mainDepthTarget); // バリアは自動計算
}
void MainOpaquePass::Execute(const RenderContext& rc) {
        drawManager->DrawOpaqueObjects(); // 描画コマンドの積み込み
}
```

Descriptor Indexingによるバインドレス化

【要件と課題】 多種類のガレキ（鉄骨、コンクリート、岩石）や敵機を描画する際、オブジェクトごとに異なるテクスチャを割り当てる必要があり、ディスクリプタテーブル切り替えによるステート変更負荷がインスタンシング描画の課題となっていた。

【比較と選定理由】 テクスチャアトラス化は解像度制限や余白の調整が必要となるため不採用。DirectX12 (Shader Model 6.6) の**Descriptor Indexing**を導入し、GPU上で動的にテクスチャを参照する手法を選択した。

【実装の工夫と代償】 全テクスチャをグローバルSRVヒープに集約し、ルートシグネチャに1度だけバインド。各オブジェクトからは整数インデックス（uint textureID）のみを渡し、HLSL側で gTextures[index].Sample(...)として動的解決させた。

【定量的な成果】
異なるテクスチャを持つオブジェクトが混在しても、ステート変更コストを抑えて一括描画可能となり、描画コマンドの効率化を達成した。

```hlsl
// Bindless.hlsli: SM6.6非有界テクスチャ配列
Texture2D<float32_t4> gTextures[] : register(t0, space1);

// Object3d.PS.hlsl: マテリアル番号による動的参照
PixelShaderOutput main(VertexShaderOutput input) {
    // オブジェクトごとに再バインドせず、配列番号から参照
    float4 texColor = gTextures[gMaterial.textureIndex].Sample(gSamplerWrap, uv);
}
```

非水密・薄型メッシュに対応する表面サンプリングフォールバック

【要件と課題】 前作ではビル等の単純な立体をボクセル化していたが、本作で多様な敵モデル（ドローンやスナイパー機）をボクセル化する際、**薄板ポリゴンや非水密（穴のあるオープン）メッシュにおいて従来の 3軸レイキャスト内外判定が失敗し、ボクセル生成数が0個になってしまう問題** に直面した。

【比較と選定理由】 アセット側をすべて完全な水密立体（閉じた形状）に制限するのは制作上の制約が大きすぎる。そこで、エンジン側で**「レイキャスト内外判定でボクセルが 0個になった場合、表面ポリゴン重心からグリッドへ直接サンプリングする表面フォールバック処理」**を導入し、あらゆるメッシュ形状を確実に破砕可能にする手法を採用した。

【実装の工夫と代償】

表面ポリゴン重心の直接サンプリング ：3軸レイキャスト内外判定でボクセル数が0個になった場合のみフォールバックを発動。各ポリゴンの重心座標（Centroid）を計算し、ボクセルグリッドの対応セルへ直接配置。

ビット結合ハッシュによる重複セルの排除 ：同一セル内に複数ポリゴンの重心が落ちた際の多重生成を防ぐため、グリッド座標（x, y, z）をビットシフト結合したキー（uint64_t）を用い、ハッシュセットで即座に重複をスキップして破片数を最適化。

【定量的な成果】

羽や装甲などの薄板ポリゴンや非水密メッシュを持つ敵モデルでも確実にボクセル化を成功させ、アセットの形状に左右されない堅牢な破砕処理を実現した。

```cpp
// ModelManager.cpp : 薄型・オープンメッシュに対する表面フォールバック
if (result.voxels.empty()) {
    for (const auto& mesh : model.meshes) {
        // 各ポリゴンの重心（Centroid）からグリッド座標へ直接サンプリング
        Vector3 centroid = (p0 + p1 + p2) / 3.0f;
        int gx = std::clamp((centroid.x - aabbMin.x) / voxelSize.x, 0, res.x - 1);
        ...
        result.voxels.push_back(newVoxel); // 確実にボクセル破片を生成
    }
}
```

